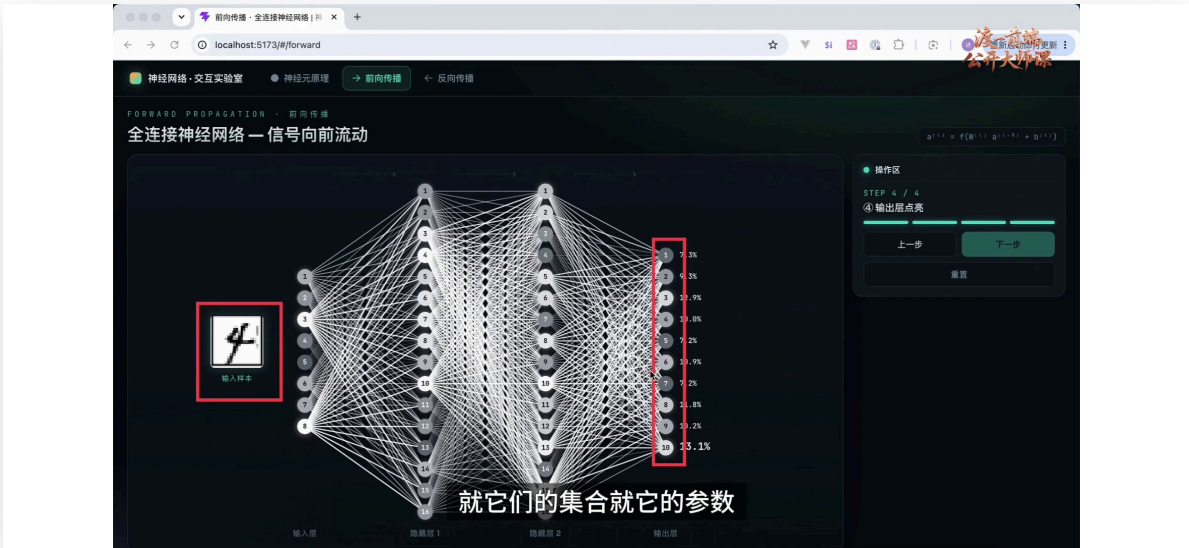


4.Agents底层逻辑之梯度下降 - 图文解说

一张关键画面对应一段完整解说，按原视频时间推进。

01. 从前向传播到自动调整参数 (00:00-01:40)



场景 1

完整内容

其实你学完之后会发现，整个神经网络就是在做一道一道的数学练习，只不过这个习题非常非常庞大。

上节课我们已经知道：在神经网络当中，前向传播从输入开始，一层一层进行运算，最后运算到输出层，就会得到一个结果。分析这个结果，我们可以看出哪个结果的几率最大。现在输入一个样本“4”，字幕显示它“认为是 10”（沿用此前的神经元编号），结果已经错得离谱，完全不能看。

接下来的问题就是上节课说的：当输入固定时，是什么在影响这个结果？是不是参数？你脑袋里面要想到“参数”是什么意思。来回答我：参数是什么意思？整个神经网络所有的权重和所有的偏置，它们的集合就是参数。正是这些参数在影响结果。

要改变结果，使它正确，是不是就要调整这些参数？那么怎么调整呢？这里涉及几万个参数。我们这个小例子，把输入展开，会有很多输入。之前算过，有两万多个参数。这么多参数，不可能像之前单个神经元那样手动调整。你想象一下，两万多个参数，该怎么调？这很

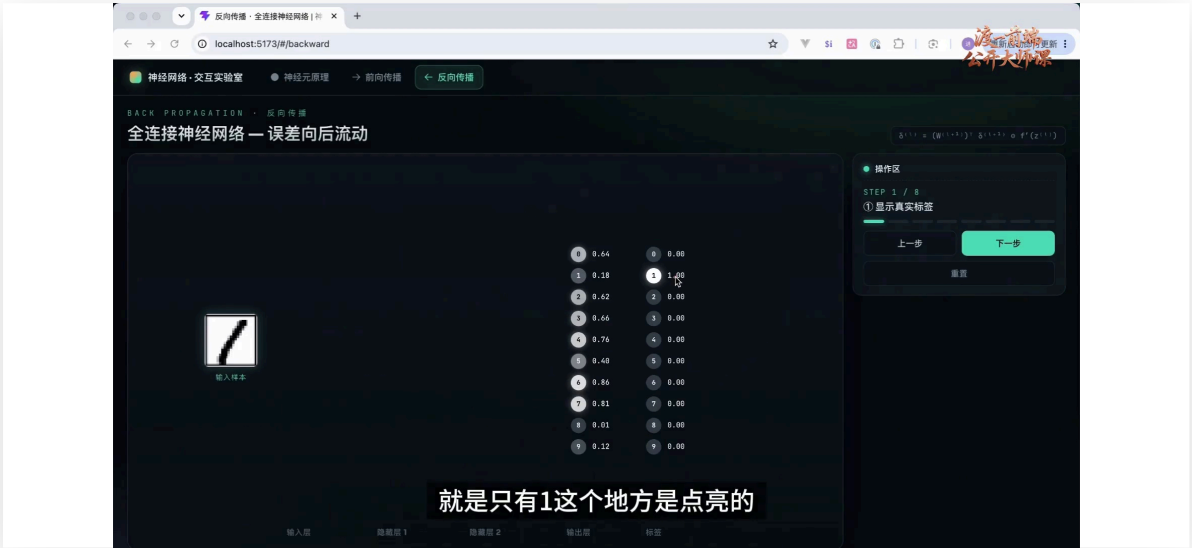
麻烦，所以我们希望计算机能自行调整。

接下来讨论的就是：如何根据结果，让计算机自行调整参数，使结果趋向正确。这就要讲到反向传播。我们从头开始：现在输入一个样本，这个样本明显是数字“1”，人一眼就能看出来；但计算机只做了一大堆数学题，算出了一个乱七八糟的结果。

画面说明

演示界面展开多层全连接网络，输入是手写数字“4”，红框标出输入与输出位置。大量连线对应权重，神经元还各有偏置；这些量的集合就是参数。开场另一候选画面的字幕写“是 he 认为是10”，这里沿用的是旧神经元编号，不能把它当成新增的数字类别。

02. 预测值、真实值和标签 (01:40-03:24)



场景 2

完整内容

我们假设每个输出都是 0 到 1 之间的数字，数字越大，表示它认为相应结果的几率越高。看当前最大的数：先看成了 0.86、数字“6”，再更正为 0.76、数字“4”。但图片明明不是“4”。完全随机的权重、完全随机的偏置，导致了这样一个随机结果。

现在的问题是如何调整输出。要调整，我们得告诉计算机，正确的值应该是什么样。可以通过数据标注来做。之前讲 AI 分类时，大家看过标注图片：人类识别出图片就是“1”，现在的结果明显不对。计算机可以用自动化程序读取这个正确结果。

神经网络算出来的结果，叫作预测值。接下来给它一个真实值，真实值也叫标签。预测值和标签之间有差距。这里的标签是一个典型的“1”。为了让大家看清楚，我把编号改了一下：之前按神经元顺序编号，现在用它代表的实际数字编号，第一个神经元代表“0”，第二个代表“1”，一直到“9”。编号从哪里开始，本身并不重要。

正确结果应该是“1”。这就意味着编号为“1”的神经元应该完全点亮，达到最大值 1。这里的输出范围是我们人为规定的：让神经元输出在 0 到 1 之间。要做到这一点，有各种数学工具；感兴趣可以问 AI，有哪些工具能把数值压缩到 0 到 1。

所以给它的正确结果是：只有“1”这个位置点亮，其他位置全为 0，全部是暗的。我们期望神经网络通过调整参数，接近这个正确结果。

画面说明

画面把预测值和标签并列：预测依次为 0.64、0.18、0.62、0.66、0.76、0.40、0.86、0.81、0.01、0.12；标签只有编号 1 为 1，其余为 0。画面实际最大值是编号 6 的 0.86。口述先说它，随后改称编号 4 的 0.76，后一个说法与可见数值不一致；正文保留这个口头更正过程，不把它当成正确的最大值判断。

03. 用损失函数量化预测与标签的差距 (03:24-05:34)



场景 3

完整内容

输出层的预测值和标签肯定有差距。第一个问题是，怎么衡量差距到底多大？我们需要量化的衡量。这个数字叫损失值，课上也称损失度；用于衡量它的东西叫损失函数，也有教材翻译成代价函数。

无论叫代价函数还是损失函数，计算出来的结果都是一个数字，也就是损失值。在这里采用的损失定义下，它不小于 0；最小值 0 表示没有损失，完全正确。不过，把它调整到 0 很困难，非常困难。我们尽量让它小就行。

损失值怎么算？不同神经网络可以采用不同的计算方式。课件列了一种非常简单的办法。看图：把一对预测值和标签相减，再求平方；下一对也相减、求平方；其他各对同样处理。平方是为了让结果不出现负数，前者减后者或后者减前者，平方之后一样。然后把这些平方全部加起来，就是演示里所用的损失度。界面显示这个例子的损失为 4.0498。

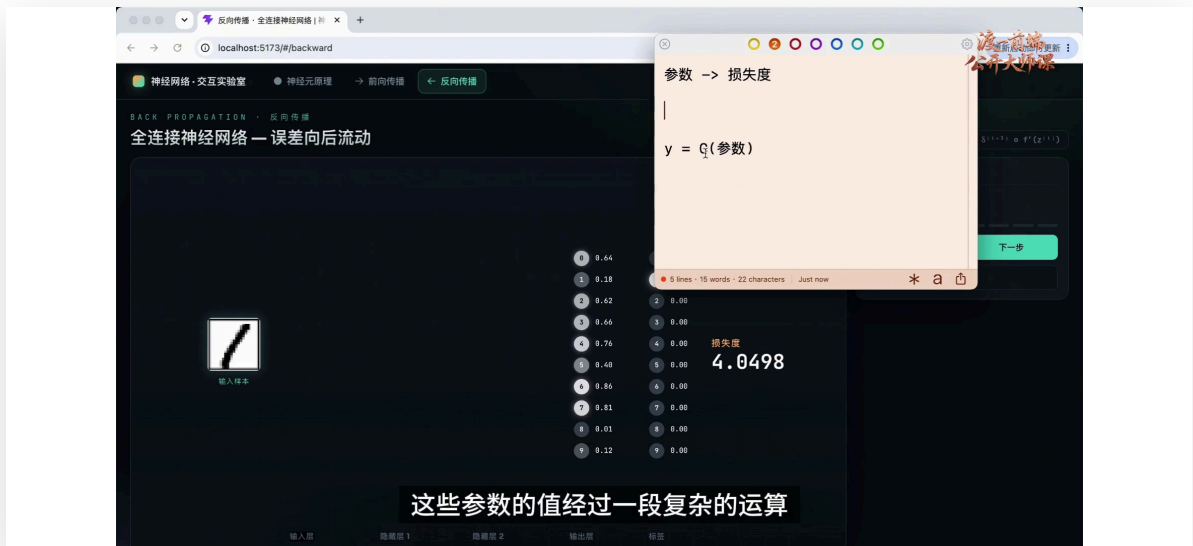
总之，我们可以计算出一个损失度。接下来进入稍复杂的环节：拿到它以后怎么办？后面的内容，如果听着有困难、脑袋晕了一下，可以暂停，好好想一想，因为背后有数学原理。但我不会把数学讲得很深，尤其微积分、偏导数这些，不展开推导。

我们前面讲概念，是为了能日常交流，能听懂别人蹦出来的词。面试时完全听不懂，会觉得心里没底；平时刷短视频，也容易觉得神经网络、模型、AI 很神秘。这里就是要消除这个神秘感。学完以后你会发现，神经网络是在做一道一道数学练习，只不过习题非常庞大，没有那么神秘。拿到损失度之后，我们的目标当然是减少它。

画面说明

红框逐对标记预测值与标签，右侧显示损失度 4.0498。口头演示采用“差值平方后相加”；末尾课件另显示带 $1/m$ 的平均平方差公式。两种写法差一个平均因子，不能把课件公式和此处的界面数值混为同一计算约定。

04. 把损失看成参数的函数 (05:34-07:42)



场景 4

完整内容

看着：在输入固定的情况下，标签是不是也固定？想一想。输入是“1”，标签就只能是那个样子，因为它是人类标注好的。所以输入固定，标签也固定。

要改变损失度，是不是只能动输出？但输出能直接动吗？不能直接动。什么影响输出？参数。所以现在什么影响损失度？我们可以认为是参数在影响损失度。这个逻辑要想清楚：输入和标签都固定，要改变损失，只能改变输出；要改变输出，只能改变参数。因此参数间接影响损失度。

参数和损失之间一定存在某种关系，只是关系太复杂。这个小网络有两万多个参数；讲到更大的模型时，我用一万多亿、约 1.5 万亿参数来举例，强调计算规模会大得多。到底每个参数怎样影响损失度，关系非常复杂。

数学里遇到这种情况，可以用函数来记：有一个 x ，经过一段非常复杂的计算，得到 y ，就说 y 与 x 有关系。具体关系是什么，即使暂时不展开，也可以写成 $y = f(x)$ 。在损失这里，我们用 cost 这个单词，用 C 表示这个函数。传入 C 的是参数；你心里要知道，这个“参数”实际是一整组数值，并不一定只有一个。

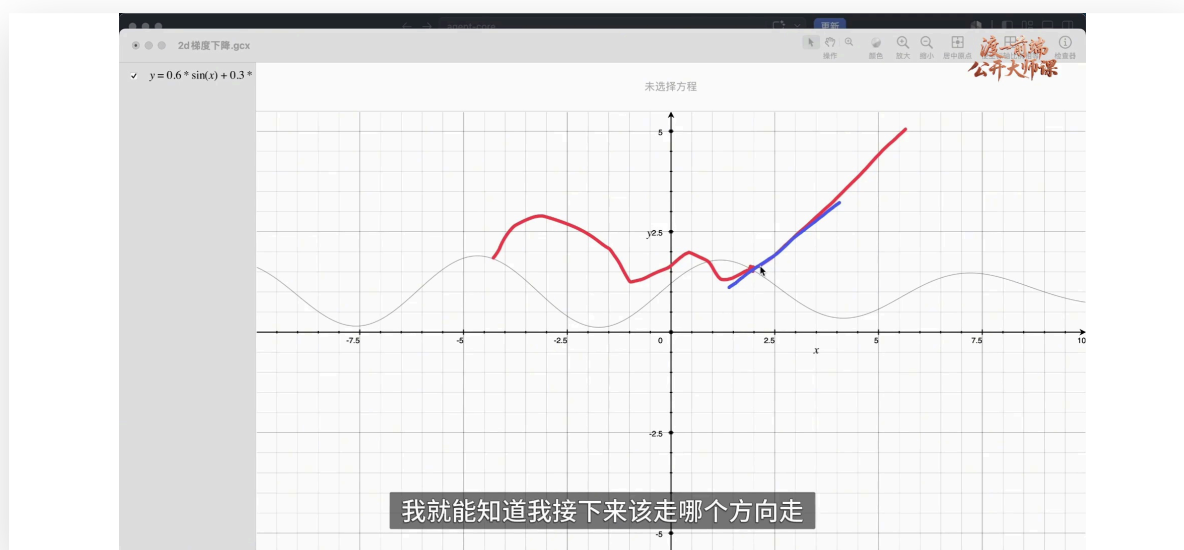
这一组参数经过复杂的运算，最后得到损失 y 。把它写出来有什么意义？我们的目标就是考虑怎样改变参数，使 y 减少，也就是让损失减少。

现在先考虑最简单的情况：假设只有一个参数，而不是两万多个。一个 x 经过复杂函数得到一个 y ，应该怎样改变 x 才能让 y 减少？接下来用一张示意图来看。

画面说明

便笺写出“参数 $\rightarrow$ 损失度”和“ $y = C(\text{参数})$ ”，浏览器仍保留样本、预测、标签及损失。箭头对应输入固定时参数影响输出、输出影响损失的推导。

05. 一个参数：由当前点的切线确定方向 (07:42-10:23)



场景 5

完整内容

假设把函数画在二维坐标系里，横坐标是 x ，纵坐标是 y 。函数很复杂，画出来可能是弯弯曲曲的一条线。现在把整个神经网络简化成只有一个参数， x 就表示这个参数。

参数最开始是随机的，所以不知道它会随机落在哪个位置。假设它落在图里的这个点。再提醒一次，如果这里想不明白，可以停下来好好想： x 表示参数；给定 x ，经过复杂计算，就能算出 y 。至于整个 C 函数的曲线长什么样，我们暂时不知道。我这里随便写了一个示意函数，你也可以想象别的曲线穿过同一个点。

这条曲线可以是这种形状，也可以是另一种形状。按这里的损失定义，损失最小是 0，所以图中的损失不能小于 0。总之，虽然不知道全局曲线是什么样，它经过当前这个点。

有人问：怎么保证它光滑？这背后有数学原理。这里先用“几乎光滑”来理解，不能说百分之百；个别不光滑的地方可以通过相应手段处理。这些细节先不深究。这里以理解概念为主，不作严格数学展开。

现在已知当前点在曲线上，怎么调整 x 才能让 y 减少？虽然不知道整体曲线，但如果能求出当前点处的斜率，也就是切线，就可以判断方向。我先说“斜率”，是为了让没有学过导数的同学也能理解；实际上，这条切线可以通过微积分中的导数求出来。

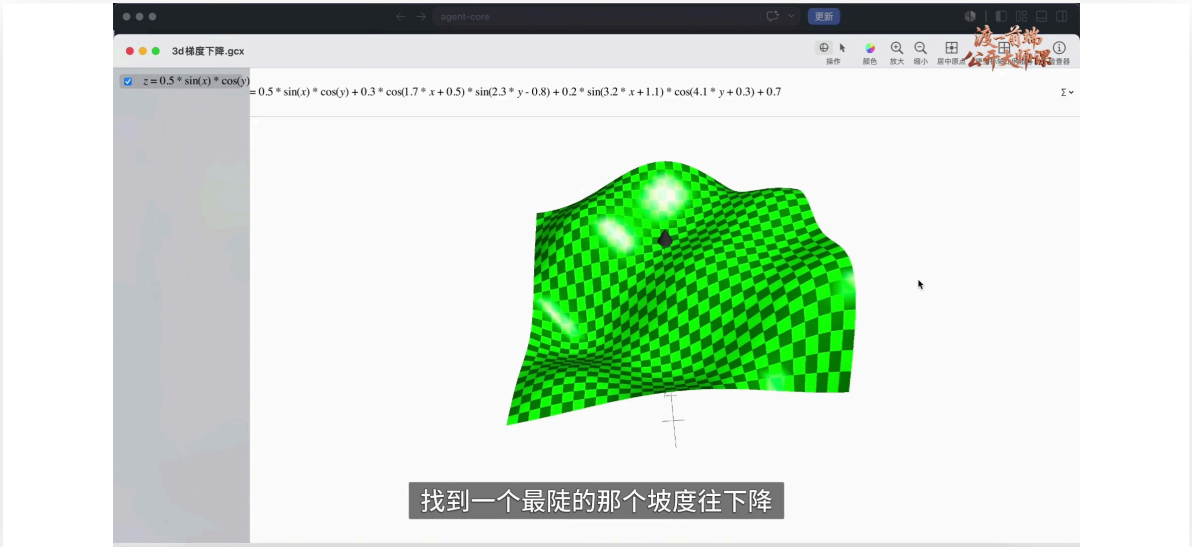
只要知道切线长什么样就行，不必先看清全局曲线。图里切线向右下方倾斜，就应该往右走，让 x 增加， y 就会在局部减少。反过来，曲线如果是另一种形状，当前切线向另一边倾斜，就该往左走。

所以，即使整体函数复杂，只要知道当前切线，就能知道接下来往哪个方向走。至于走多远，这里还不知道；现在先解决方向问题。这是一个参数的情况。

画面说明

二维坐标上有原来的灰色曲线、手绘红色替代曲线及蓝色局部切线。手绘线展示不同全局形状也可经过当前点；当前切线用于判断局部往左还是往右。

06. 两个参数：在曲面上寻找下坡方向 (10:23-12:29)



场景 6

完整内容

如果有两个参数，就不能只画二维平面中的曲线了，要用三维曲面。这里用的是 Mac 系统里的 Grapher 软件。它是教学用具，用来展示形状；如果你的 Windows 环境打不开这个课件，也不影响理解演示。

看三维曲面：一条轴是 x ，另一条轴是 y ，竖直的 z 轴表示损失度。让损失度最小是什么意思？我们旋转一下视角，想象自己站在像山一样的曲面上。脚下两个方向分别是 x 和 y ，竖直的上下方向是损失度。

想让损失度减少，是不是就要往下坡走？那往哪个方向走呢？可以想象：伸手不见五指的黑夜，你站在连绵起伏的群山中，不知道自己在什么位置，也看不见整座山的形状。你可以通过脚底感受哪个方向的坡度最陡，沿着最陡的下坡方向走一点，就在下降。下降，就意味着损失度减少。

这就是梯度下降的形象理解：找到最陡的下坡方向，往那个方向走。这里先把梯度下降作为一种整体思路来理解，还没有展开具体的更新公式。不是先要看清整个函数，而是想办法找到当前位置最陡的下降方向。

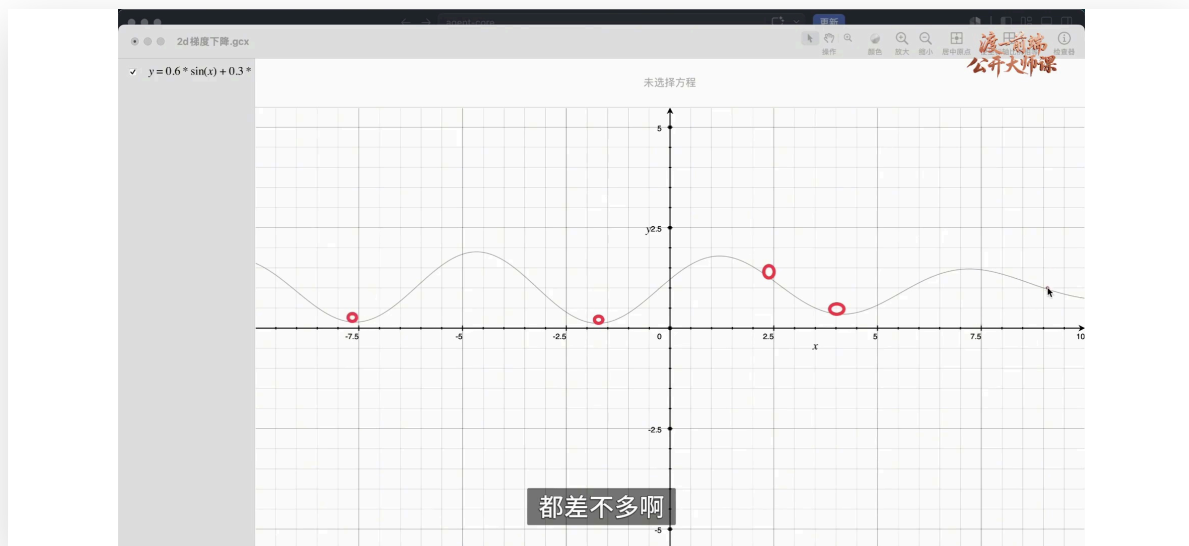
刚才只是两个参数。实际这个例子有两万多个参数，也就是高维空间，画不出来。但可以想象，在高维空间里，也寻找这样的下降方向。具体怎么算、怎么走，后面才开始讲，不着急；先有一个整体感觉。

通过调整参数，让损失下降，就是这里所讲的梯度下降。先记住：沿着当前最陡的下坡方向走。接下来再讨论如何实现。

画面说明

Grapher 显示带绿色棋盘纹理的起伏曲面，可以看到多个山峰和凹谷。 x 、 y 表示两个参数，竖直方向对应损失；旋转视角用来解释黑夜下山比喻。

07. 局部最优、全局最优与山谷比喻 (12:29-14:26)



场景 7

完整内容

现在能理解“梯度下降”这个概念了吗？以后听到这个词，可以想到刚才的图。无论二维曲线还是三维曲面，感觉都像一个小球往下滚，滚到凹槽里。是不是损失就小了？不要一开始就追求损失一定为 0，先尽量滚到一个坡底。

有同学会说：不对，最低的位置应该是另一个地方！对，现在这个位置可能只是局部最优，另一个位置才是全局最优。你的道理是对的：顺着当前位置滚下去，可能只能到一个局部最优位置，并不保证到达全局最优。

这里用不断延伸的曲线来说明困难：你很难把整个空间找完，所以这节课先不追求一定找到全局最优。从实际训练的直观经验来讲，局部较好的解也可能已经很不错；通过对神经网络作精心设计，不同地方的局部结果可以比较接近。这里不是说随便一个局部最优都一样好，而是在解释为什么先寻找可用的局部下降结果。

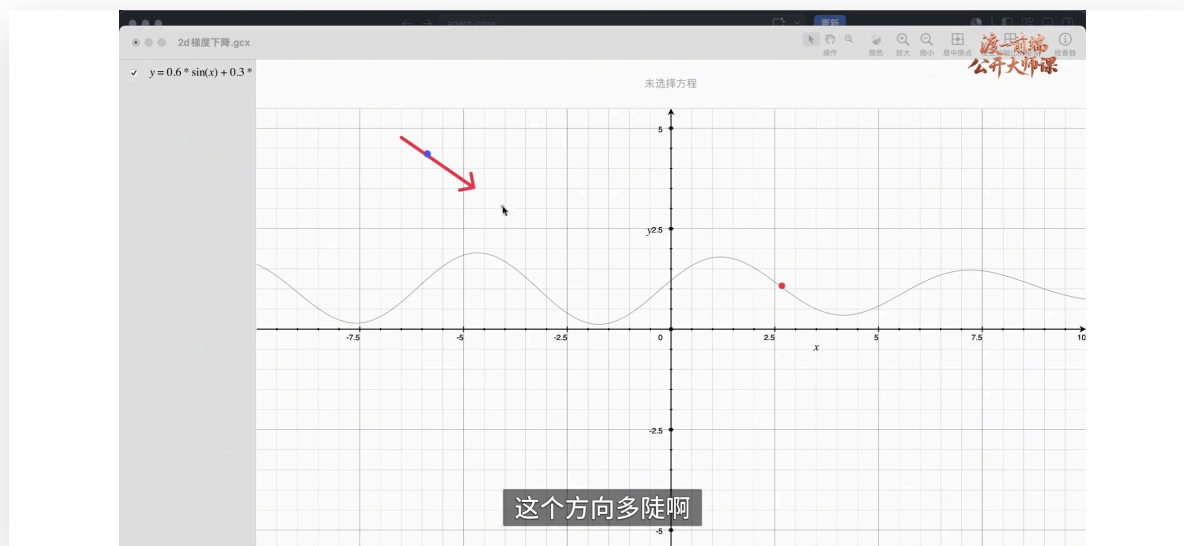
三维空间也一样。旋转、缩小图像后，可以看见曲面继续延伸。初始的 x 、 y 如果落在这个位置，小球往下滚，可能只能滚进附近这个凹槽。它是一个局部结果，不一定是整个空间里最低的地方。不要把眼前坡底和全局最低点混在一起。

如果陷入一个不太好的局部位置，现在的算法也会用一些额外手段。比如加入一些噪声，让它有机会振动出去。这里不展开这些方法；整体上先用“找到一个凹槽滑下去”来形象理解下降过程。具体怎么做，接下来继续看。

画面说明

灰色函数曲线上圈出多个红色位置，包括不同凹谷与一个较高位置。图中各个谷底的高度不同，用来区分局部较低点与全局最低点。

08. 学习率：方向之外，还要控制每次走多远（14:26-16:27）



场景 8

完整内容

刚才还有一个点没说，就是学习率。我们回到二维图。假设已经算出了当前点的切线，也知道应该往哪个方向走。那要走多少？一大步，还是一小步？这由学习率来控制。

学习率是人为设置的量，可以先把它理解为步幅、步长：方向知道了，还要决定走多远。为什么需要这个东西？因为我们希望每一步不要走得太大，先少走一点。

再想象一下黑夜中的群山：你看不见远处，只能用脚轻轻探出一个下坡方向。方向对了，你敢一下跨很大一步吗？当然，数学里的点没有生命，不怕掉下去。真正怕的是：你只知道当前斜率，并不知道后面整条曲线长什么样。

画一个极端例子。人在这个点，切线很陡，觉得往这个方向跨一大步就好。可是曲线可能很快又弯上去了。你一步跨过去，反而落到了上坡的位置，损失变大。懂这个意思吧？因为不知道后面是什么情况，先一点一点地走。在实际应用中控制步幅还有其他原因，这里先建立这种直观。

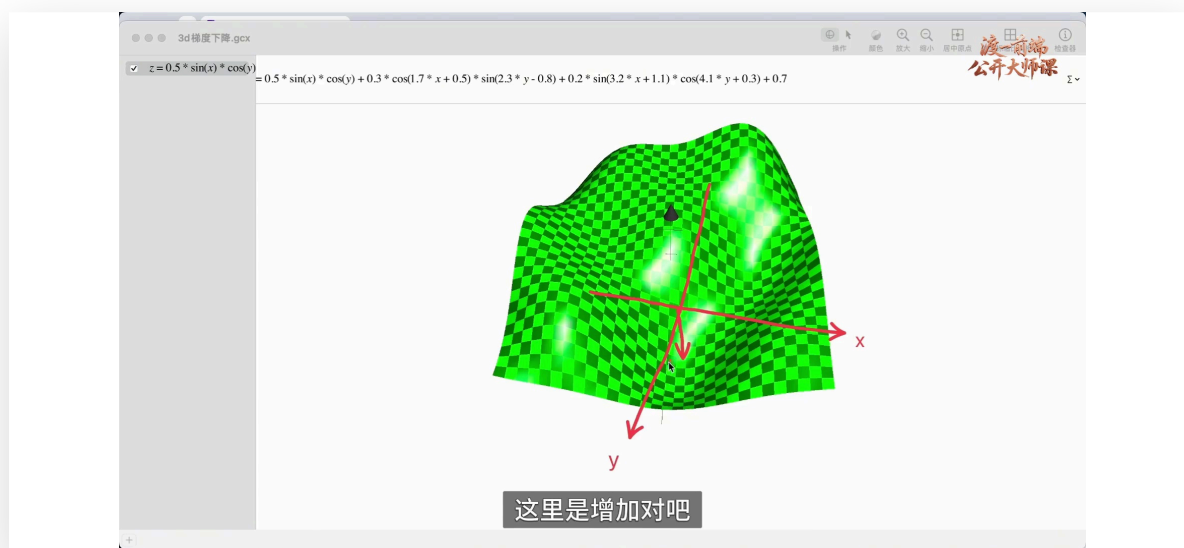
所以有两个概念：梯度下降，解决沿当前点最快下降方向移动的问题，也就是朝负梯度方向，让函数值减少；学习率，控制每次下降的幅度。不是一次下降就结束，而是要不断下降，每次先走一点。

还有一个概念需要清楚：在三维曲面上，当你往某个方向走时，这个方向在不同参数坐标轴上的分量可能不同。接下来就看这个问题。

画面说明

二维曲线的当前点上方画出红色下降箭头，说明已知方向后仍要控制步幅。大步跨过谷底是口述补充的极端示例；这张选图展示方向本身。

09. 下降方向在各参数上的分量 (16:27-18:02)



场景 9

完整内容

仍以两个参数的曲面为例。假设当前位置的坐标轴是 x 和 y ，图中箭头就是下降最快的方向。把这条方向分解到 x 、 y 两个轴上，就能看到： y 这个参数可能要改变得多一些， x 改变得少一些。

图中一开始顺口说了“ y 下降”，随后根据坐标方向更正为“ y 增加得多一点”。关键不是所有参数都要一起变小，而是沿所需方向，有的增加，有的减少，并且改变的大小不同。画出两条投影：这一段对应 x ，那一段对应 y 。最终目的，是改变 x 和 y ，也就是改变参数。

为什么强调这个？因为最后计算“怎样改变参数，才能使损失减少”时，你会得到各参数不同的变化量。有的动得多，有的动得少。刚才这个示意方向中，y 变得多一点，x 变得少一点，合起来才构成所需的最陡方向。

现在只是宏观理解，还没有涉及具体运算。三个参数再加上损失，就需要四维空间；四个参数再加上损失，就需要五维空间；两万多个参数就没法直接画出来。但道理是同一个道理。理解了这个，梯度下降的直观部分就讲完了。

接下来，如何实现它？说了半天“找坡度”，到底怎么找？这就要进入具体计算——反向传播。我们用反向传播来计算实现下降所需要的信息。

画面说明

绿色三维曲面上手绘红色 x、y 方向和下降方向。各坐标方向的分量长短不同，演示中随坐标方向更正“增加/减少”，重点是不同参数的变化大小不同。

10. 从输出层的期望开始反向传播 (18:02-19:37)



场景 10

完整内容

反向传播和梯度下降是什么关系？按这里的讲解，梯度下降给出目标：沿最陡的下坡方向走，使损失减少。反向传播用于算出实现这个目标所需要的调整信息。看它怎么做。

现在这边是输出层，也就是预测值；另一边是标签，也叫真实值。观察第一个神经元：期望它为 0，但它现在是 0.64。那希望激活度减少还是增加？希望减少。下面一个神经元，真实值是 1，但结果为 0.18，就希望它增加。

可以逐个算出这种“期望”。例如： $0 - 0.64 = -0.64$ ，希望减少 0.64； $1 - 0.18 = 0.82$ ，希望增加 0.82。界面里向上的箭头表示期望增加，向下的箭头表示期望减少。箭头有大有小，表示期望的力度不同。

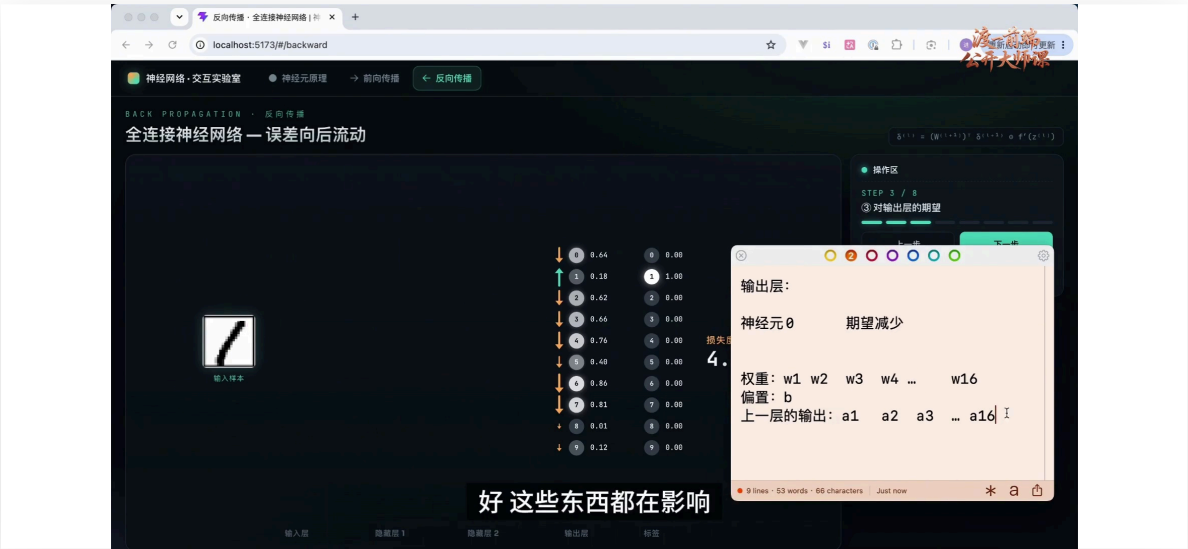
比如输出已经是 0.01，非常接近目标 0，期望它调整的幅度就很小，动一点就行。对每个神经元，都有不同方向、不同大小的调整期望。这是输出层的情况。

接下来思路要跟上，不然容易绕晕。我们逐个看输出层神经元，先拿第一个，也就是编号“0”的神经元来看。

画面说明

输出层与标签并列，输出层边上出现不同长度、不同朝向的箭头，代表相对标签需要增加或减少的期望。0.64 对应目标 0，0.18 对应目标 1。

11. 一个输出神经元受哪些量影响 (19:37-20:53)



场景 11

完整内容

对于输出层第一个神经元，我期望它减少，而且能算出具体希望减少多少。总之，要让它的激活度下降，怎么做？既然已经到了输出层，如果它能达到目标，输出就对了。但它的激活度由什么决定？

首先是权重。上一层有 16 个神经元，所以这个神经元有 16 个输入权重： w_1 、 w_2 、 w_3 、 w_4 ，一直到 w_{16} 。它们都影响激活状态。除了权重，还有什么？还有一个偏置 b 。

再想一想，还有没有？还有上一层的输出！这里是 w_1 乘 a_1 、 w_2 乘 a_2 ，等等， a_1 、 a_2 、 a_3 一直到 a_{16} ，都是上一层的输出。

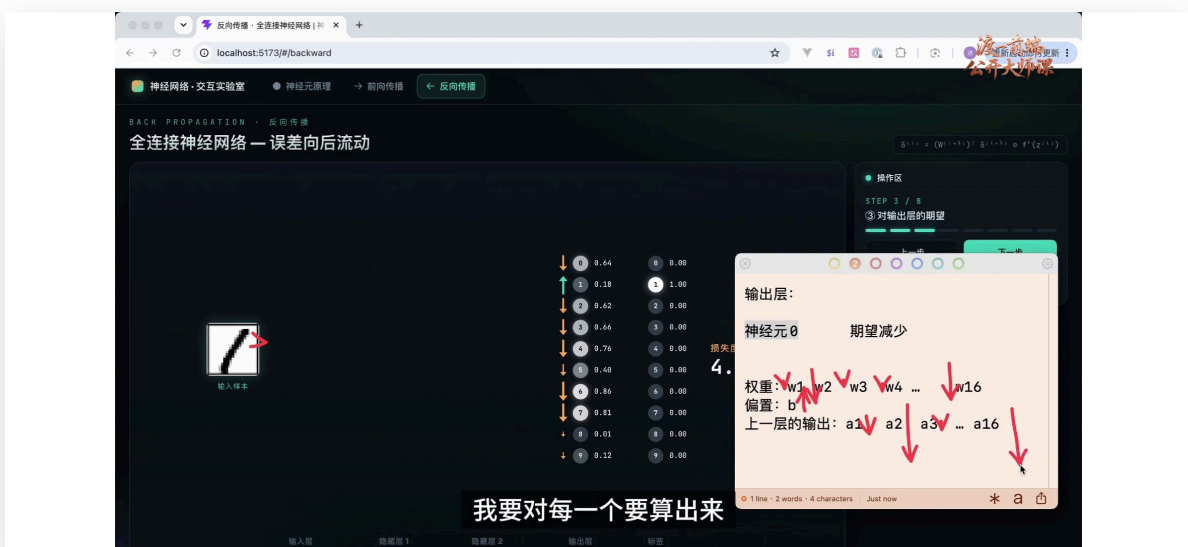
有人可能会问：“上一层输出不是不用管吗，输入不是固定的吗？”要区分。整个网络最初的输入是固定的，但这里上一层并不是输入层。隐藏层的输出会受它前面的参数影响，不能直接说它固定。

所以，权重、偏置、上一层的激活值，都影响当前这个神经元的亮度。现在期望它减少，就要考虑怎样改动这些量，以及哪些量对激活状态影响多、哪些影响少。

画面说明

便笺列出输出层神经元所依赖的三组量：权重 $w_1 \cdots w_{16}$ 、偏置 b 、上一层激活 $a_1 \cdots a_{16}$ 。左侧仍是输入样本、预测与标签，便于区分原始输入和隐藏层输出。

12. 用偏导和链式法则计算各项调整量 (20:53-23:01)



完整内容

现在要计算：怎样改动这些量，才能让当前神经元的激活度减少？哪些应改得多一些，哪些少一些？它们分别对激活状态有多大影响？

举个示意： w_1 减少一点点， w_2 减少多一点， w_3 、 w_4 、 w_{16} 各减少一点；偏置 b 可能需要增加； a_1 减少一点， a_2 减少多一些， a_3 减少一点。这里的箭头是用来表示方向和大小的示意，并不是正在给出这组网络的真实计算结果。

对每个量，都能算出需要怎样变。怎么算？会用到微积分，主要是求偏导。偏导的意思，可以先理解为：当有多个变量时，固定其他变量，看某一个变量如何影响函数结果，再对它求导。

这里还会用到链式法则。你平时刷视频听到“链式法则”，它本质上是微积分里的概念，不是神经网络独有的词。有兴趣可以继续了解，但这节不是数学课，先知道它能把每一项计算出来。

我这里只画箭头，表示希望亮度减少；真正算的时候， w_1 要改多少、 w_2 要改多少，能得到具体数值。比如一个减少 0.01，一个减少 0.12，一个减少 0.05。这些数字是说明“可以算出数量”，不只是笼统知道方向。

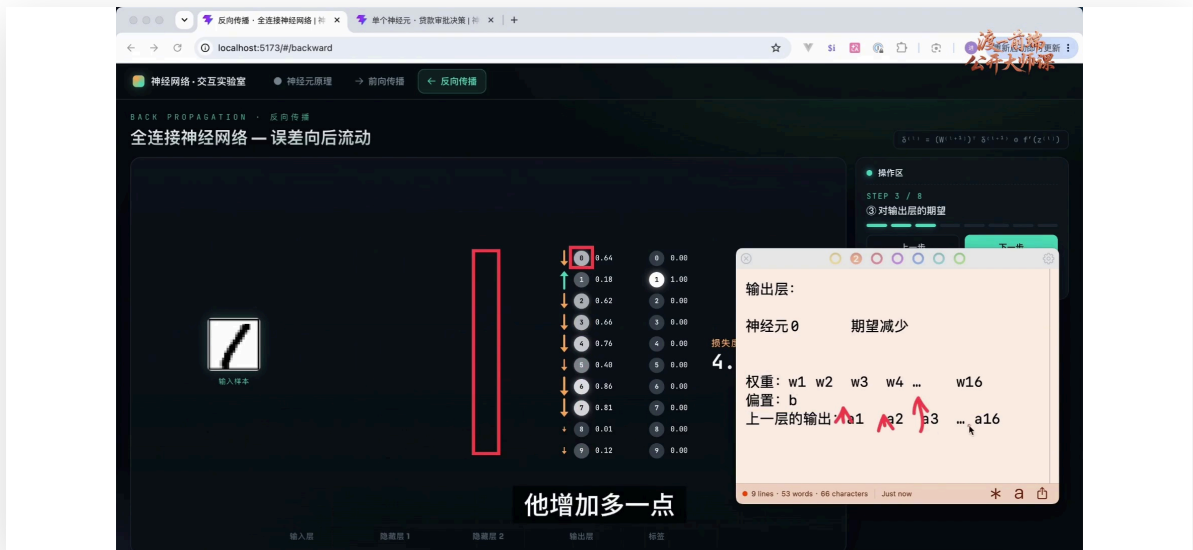
现在先计算，并不立即逐个改动，等全部弄完以后一起操作。再回顾目标：要算出整个神经网络每个权重和偏置应该增加或减少的量，重新调整参数。

刚才只看了第一个输出神经元。第二个也是同样道理：期望它增加，就按同样的方法计算它每个权重和偏置需要怎样调整。第三个、其他各个也是如此。这样，这一层所有神经元的权重和偏置调整量，就都能算出来。训练模型就在做这些计算，背后有数学工具支持。

画面说明

便笺各权重、偏置及上一层激活旁画出红色箭头，大小、方向各不相同；这些是讲解用示意，不是实际梯度数值。

13. 不能直接拨动上一层输出，就把期望传回去 (23:01-24:15)



场景 13

完整内容

到这里还有事吗？还有：上一层的激活值怎么办？权重、偏置是参数，像旋钮一样可以调。我们能算出这个加 0.2、那个减少多少，再去调整；但上一层的输出本身没有一个可以直接拨动的旋钮。

这就开始出现反向传播的关键。每个神经元不只是一要调整自己的权重和偏置，它还会对上一层的输出提出“期望”。这一个神经元，对上一层所有相关输出都有期望：这个增加一点，那个多加一点，另一个少加一点，或者减少一点。

不管最终计算出来是正数还是负数，总之存在一个方向和大小。虽然不能直接改上一层输出，但可以对它有期望。这是不是类似标签对当前输出层提出的增加、减少期望？

看下一步，方向就倒回去了。这就是为什么叫反向传播：当前层计算出的需要，会传向前一层。

画面说明

便笺中上一层激活 a_1 、 a_2 、 a_3 附近标出向上箭头，主图左侧框出上一层所在位置。当前神经元既有自己的参数调整，也会把对上一层输出的期望传回去。

14. 汇总多个期望，逐层反传到第一个隐藏层 (24:15-25:59)



场景 14

完整内容

当前这个神经元，对上一层一系列神经元都有不同期望。为什么图中有多个箭头？因为输出层的第二个神经元，也对前面这些神经元有期望；其他神经元也一样。

这里的箭头大小、方向是示意，我没有拿实际数据算出来再画。它们有大有小、有上有下，只是为了展示“不同期望”。不要把示意图中的长度当成真实计算值。

这和标签对输出层的关系不同：标签与输出是一对一比较；这里则是多对多。后面每个神经元都可能对上一层的各个神经元提出期望。那上一层到底听谁的？比如一个希望它向下，另一个希望它向上，第三个也希望它向上，看起来会冲突。

但刚才说了，期望有大小。把它们加起来就行。所有针对同一个神经元的期望相加，合起来向上，就按向上；合起来向下，就按向下。多支箭头合成一支箭头。

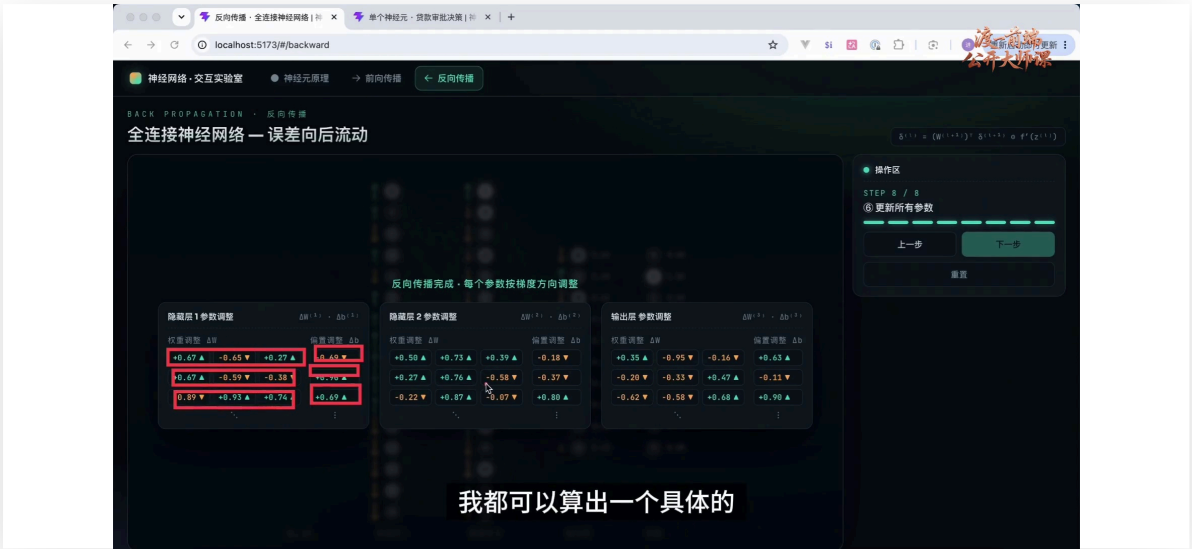
得到这一支箭头后，处理方式是不是又循环起来了？对这一层每个神经元，根据期望计算它的权重和偏置调整量，同时产生对更前一层的期望。更前一层又收到多支箭头，再相加成一个，再计算自己的权重和偏置。

一直到第一个隐藏层时，还需要对再上一层有期望吗？在这节讨论的参数训练中不需要继续调整输入，因为再前面已经是输入层了。输入来自用户、来自训练数据，是固定的。别再指望把输入改成另一个样子。第一个隐藏层只需要算出自己的权重和偏置调整量，就到这里。

画面说明

当前显示的隐藏层神经元旁有成组小箭头，输出层也保留调整箭头。多支箭头针对同一节点，体现来自下一层多个神经元的贡献需要相加。

15. 统一更新参数，再循环前向与反向传播 (25:59-27:39)



场景 15

完整内容

这样反向传播过来，先搞定输出层的权重和偏置，再搞定前面的隐藏层，再往前。所有层的权重和偏置，都能得到一个结果：某个权重要调高多少，另一个要调低多少，每个偏置怎样调整。

图里只画出一小部分，不可能把每个神经网络的所有权重全画完，但计算时是每一项都有数值。隐藏层各项汇集起来，又可以写成矩阵。整个计算过程大量使用矩阵运算，统一交给 GPU 进行并行计算和更新。

当然，这个过程中还要乘上学习率。具体公式这里不展开，但作用要知道：让调整幅度小一些，别一次调得太多，每一步走得小一点。于是机器可以自行调整这些“开关、旋钮”，就像之前单个神经元的例子那样。

调整完，相当于沿下坡方向走了一步。在这套直观说明里，就是损失减少了一些。但减少一些就够了吗？当然不够，后面还有下坡路，要继续观察、继续走。下山的路可能蜿蜒曲折：先往这边一点，走到新位置后，那边又更陡，就换个方向再走一点。

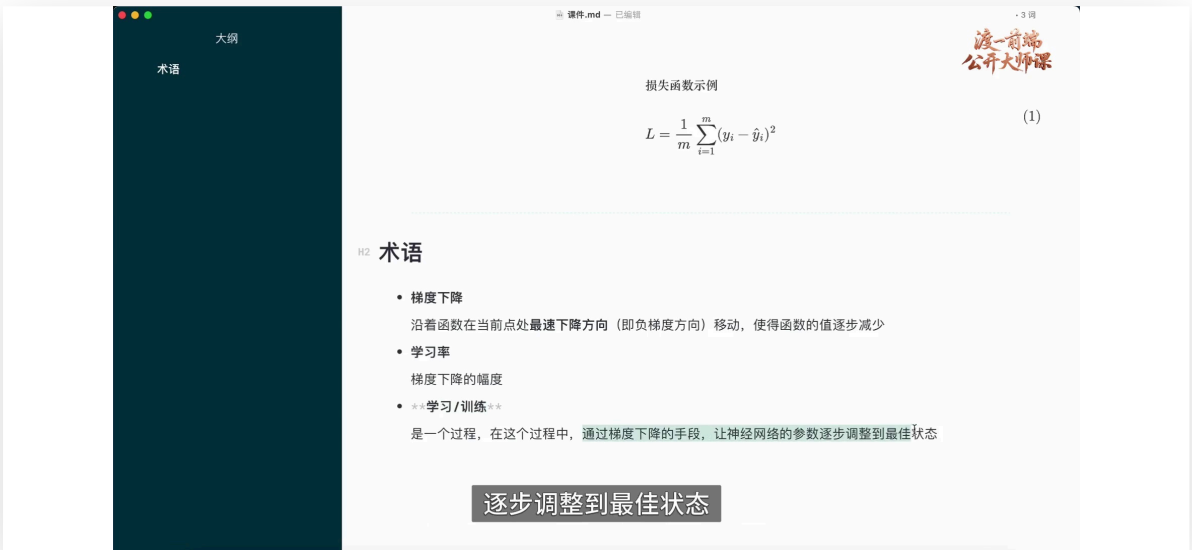
要慢慢走，才能到达某个局部较优的位置。你不知道坡底在哪里，得走一步算一步。因此整个过程要循环往复：再做前向传播，算出预测与损失；再做反向传播，算出调整信息；再更新参数。

逐渐调整，直到损失已经比较低，继续试着调整也没有多少下降空间、变化已经很小。这里把它理解为调整到位。整个过程，就是让机器不断计算并调整参数。最后我们再看“学习”“训练”这些术语。

画面说明

“更新所有参数”步骤把隐藏层 1、隐藏层 2、输出层的 ΔW 与 Δb 分成三栏。绿色上箭头和橙色下箭头表示增加与减少，例如左侧可见 +0.67、-0.65、+0.27 等示意调整值。源演示随后出现 5.0068 的损失显示，与先前 4.0498 不构成下降的数值验证；此处只能据此说明参数更新界面的结构，不能声称画面实证了收敛。

16. 学习与训练：通过反复计算把参数调到较好状态 (27:39-28:34)



场景 16

完整内容

现在可以给“学习”“训练”下一个相对准确的定义。因为没有展开数学，我们说的是原理层面的理解：它是一个过程，在这个过程中，通过梯度下降的方式，让神经网络参数逐步调整到较好的状态。

这个状态不一定最完美，但至少努力把损失降下来。不要期待机器和人完全一样。人看见一个苹果，会直接认出“这是苹果”；机器这里是靠做数学题算出来的。

算完之后，你要问每一个神经元具体承担什么作用，往往没法逐个明确解释。为什么参数这样调来调去，最后出现这个结果？我们能描述计算过程，但不等于能直观说清每个神经元各自代表什么。

这就是这里讲的训练过程：通过反向传播计算信息，使参数沿下降方向不断调整，逐渐达到较好的状态。梯度下降的核心概念，到这里讲完了。

不过神经网络还没有完全讲完，仍有问题没解决。下一节是神经网络部分最后一节，会讲具体实施时遇到的一些问题，以及由这些问题产生的一些术语。我们下节课再说。

画面说明

课件可见公式 $L = (1/m) \sum_{i=1}^m (y_i - \hat{y}_i)^2$ ，并列出了“梯度下降”“学习率”“学习/训练”。梯度下降定义强调当前点的最快下降方向，即负梯度方向；学习率写为下降幅度；训练定义强调参数逐步调整。该课件有平均因子 $1/m$ ，与前面口述平方和应区别阅读。